

COMS W4156 Advanced Software Engineering (ASE)

October 31, 2023



No Discussion of First Assessment Yet!

Several students have an extended or shifted time window, we will discuss after everyone's deadline has passed



Agenda

1. **Second Iteration**
2. From CRC Cards to Code
3. Design Principles



Upcoming Assignments

- [Second Iteration](#) due Thursday, November 30, by 11:59pm
- [Second Iteration Demo](#) due Thursday, December 7, by 11:59pm



What's New In Iteration Two?

Client app that uses your service and targets one or more designated user groups

Continuous Integration (CI) using github actions

Integration testing (run via CI)

Branch coverage (run via CI)

Static analysis bug finder (run via CI)

Remember to re-run style checking, and update unit and API tests (run via CI)

End-to-end tests for client app together with service (manual is ok)



Designated Target Audiences

- Elderly (defined for this purpose as age 75 or over)
- Eldercare workers (caregivers for elderly, often non-English speakers)
- Disabled or impaired (e.g., vision, hearing, speech, motor, cognitive)
- Healthcare workers (doctors, medical technicians, nurses, aides, etc.)
- PreK-2 (early childhood) school teachers
- First responders (police, fire, EMT)
- Maybe others with "societal benefits" - ask me *in advance*



Agenda

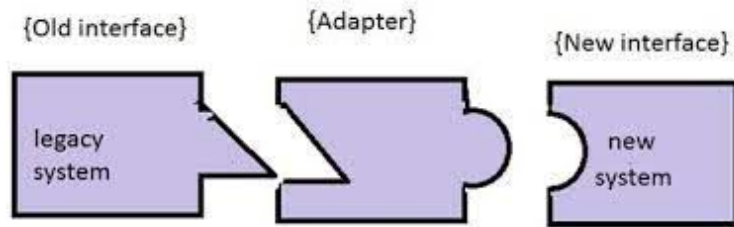
1. Second Iteration
2. From CRC Cards to Code
3. Design Principles





How to get from CRC Cards to Code

First check whether any CRC “classes” can be implemented by already available components, including libraries and services. This might require [adapters](#)



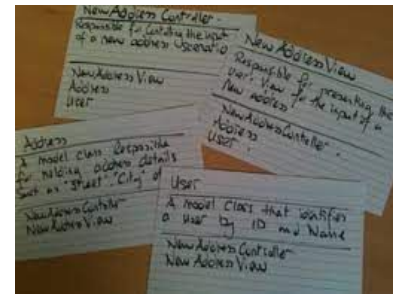
Then other CRC classes will typically be implemented by a newly written class, module, file, etc. - the programming language’s modular unit

The responsibilities correspond to the modular unit’s methods, functions, procedures, subroutines, etc., with access to the collaborators via parameters, object or class fields, global variables, shared data structures, etc.

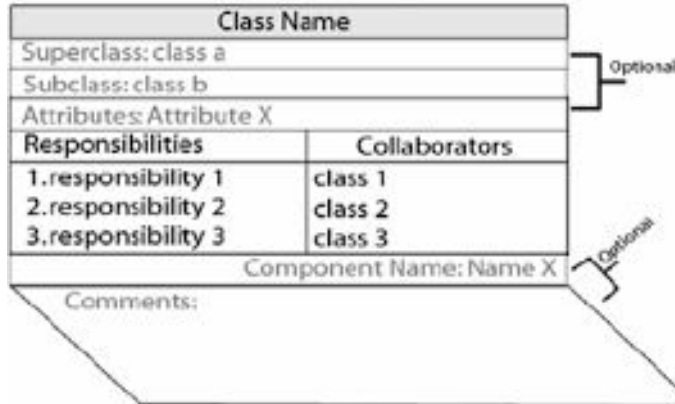
Note updating fields, global variables, shared data structures = “side-effects”

Turn over your CRC card

On the back, list:



Front Side



Back Side

Class or other component name (not necessarily same as on front, e.g., if reusing existing software)

Object fields maintained by instances of the class - data as well as references to collaborators

Class fields maintained by the class itself

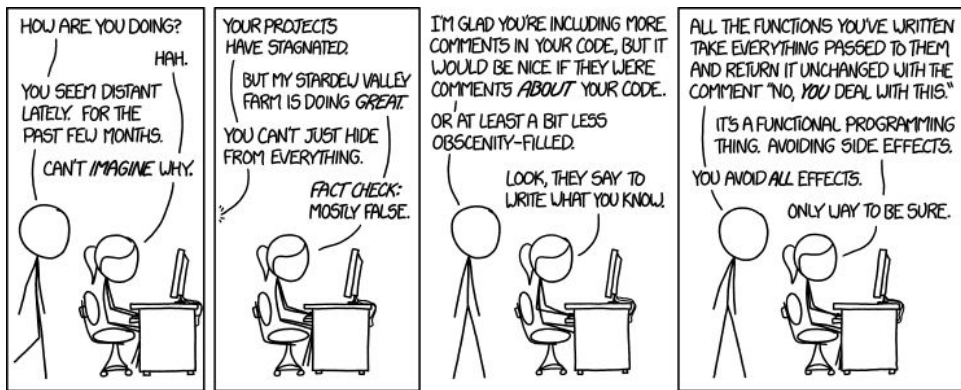
Methods that carry out the responsibilities - signature with name, parameter list, return types, *make sure to include each method's "side-effect" input/output data beyond parameters and return*

More on Side-Effects

Avoid when possible

But if your units have side-effects, then unit testing needs to consider inputs from shared state and, even more significantly, outputs to shared state

Mock shared state outside the class during unit testing, then replace with real resources during integration



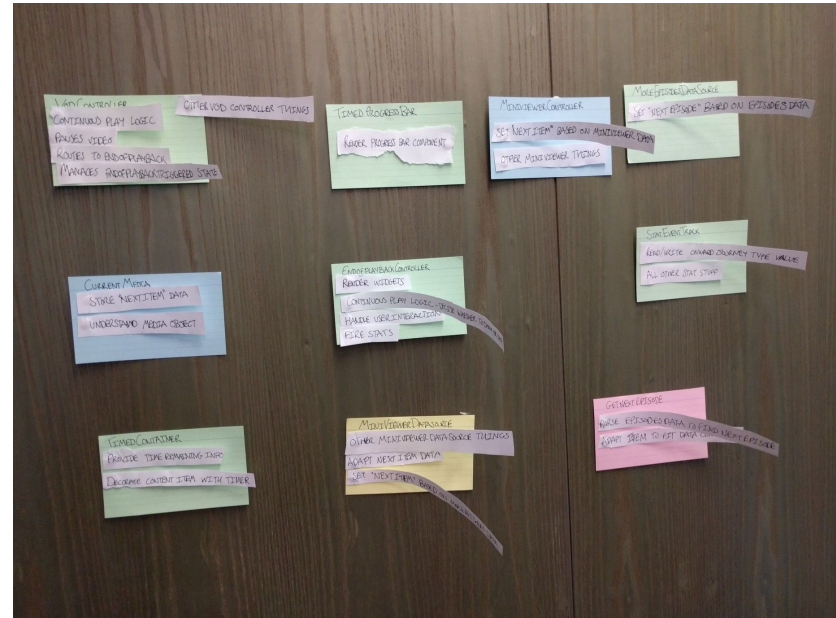
Debugging CRC Cards, Revisited

Again, animate the use cases - move the cards around on a table or board - to make sure every trigger is accounted for, every precondition is checked, every postcondition indeed becomes true, and every step in all basic, alternative and exception flows is accounted for

May help to represent actors that trigger or interact with responsibilities as “fake” CRC cards



twinkl.com



What CRC Cards Are Needed?

Name UC-8: Search and Replace

Summary All occurrences of a search term are replaced with replacement text.

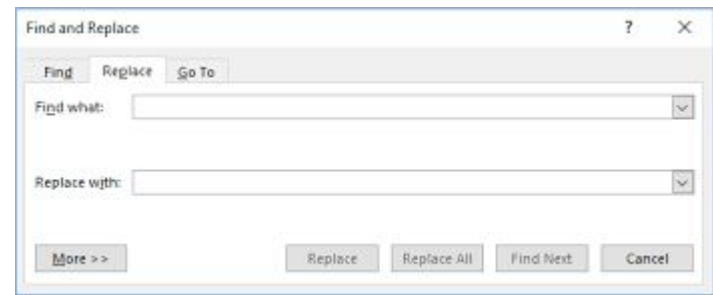
Rationale While editing a document, many users find that there is text somewhere in the file being edited that needs to be replaced, but searching for it manually by looking through the entire document is time-consuming and ineffective. The search-and-replace function allows the user to find it automatically and replace it with specified text. Sometimes this term is repeated in many places and needs to be replaced. At other times, only the first occurrence should be replaced. The user may also wish to simply find the location of that text without replacing it.

Actors All users

Preconditions A document is loaded and being edited.

Basic Course of Events

1. The user indicates that the software is to perform a search-and-replace in the document.
2. The software responds by requesting the search term and the replacement text.
3. The user inputs the search term and replacement text and indicates that all occurrences are to be replaced.
4. The software replaces all occurrences of the search term with the replacement text.



Remember there is no Global Master



When “the system” or “the software” does something, that means some class in the system does it

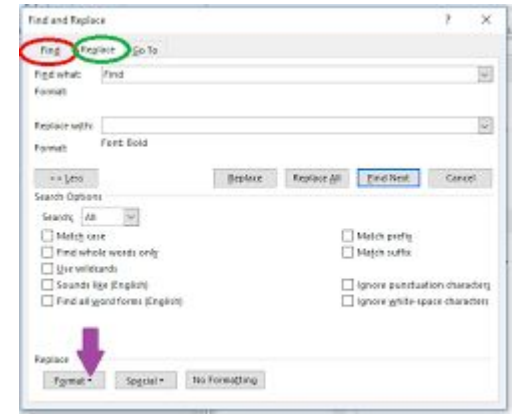
There should not be any global class that does everything and/or is related to everything else

Give up global knowledge of control and rely on local knowledge of objects to accomplish their tasks

Do The CRC Cards Cover Everything?

...

Alternative Paths



1. In Step 3, the user indicates that only the first occurrence is to be replaced. In this case, the software finds the first occurrence of the search term in the document being edited and replaces it with the replacement text. The postcondition state is identical, except only the first occurrence is replaced, and the replacement text is highlighted.
2. In Step 3, the user indicates that the software is only to search and not replace, and does not specify replacement text. In this case, the software highlights the first occurrence of the search term and the use case ends.
3. The user may decide to abort the search-and-replace operation at any time during Steps 1, 2, or 3. In this case, the software returns to the precondition state.

Postconditions All occurrences of the search term have been replaced with the replacement text.

Agenda

1. Second Iteration
2. From CRC Cards to Code
3. Design Principles



Apply Design Principles

For each extended CRC card (extended by the details on the back), consider design principles applicable to that component and/or its collaborations with other components

Design Principle = general technique to avoid bad design (or, phrased positively, create good designs)

Bad design is:

- Rigid - Hard to change the code because every change affects too many other parts of the system.
- Fragile - When you make a change to some part of the code, unexpected parts of the system break.
- Immobile - Hard to reuse some of the code in another application because it cannot be disentangled from the current application.



You Aren't Gonna Need It ([YAGNI](#)): do the simplest design that could possibly work for the functionality needed NOW

Good Software Design



- Functional - Focused on core functionality
- Robust - Resistant to changes in environment and failures
- Measurable - It should be possible to see how well the code is doing outside of a test environment
- Debuggable - Maintains logs of what its done and is doing
- Maintainable - Consistent styling, good comments. Easy to make changes to parts of the software without breaking other parts, particularly core functionality
- Reusable - Particularly if some/all of the software is realistically plausible for other products (contradicts yagni?)
- Extensible - Should be written with extension in mind (contradicts yagni?)

“SOLID” Software Engineering Design Principles

Design Principle = general technique to avoid bad design (or, phrased positively, create good designs)

1. **S**ingle Responsibility
2. **O**pen/Closed
3. **L**iskov Substitution
4. **I**nterface Segregation
5. **D**ependency Inversion



Design Principle != Design Pattern

(design patterns next week)

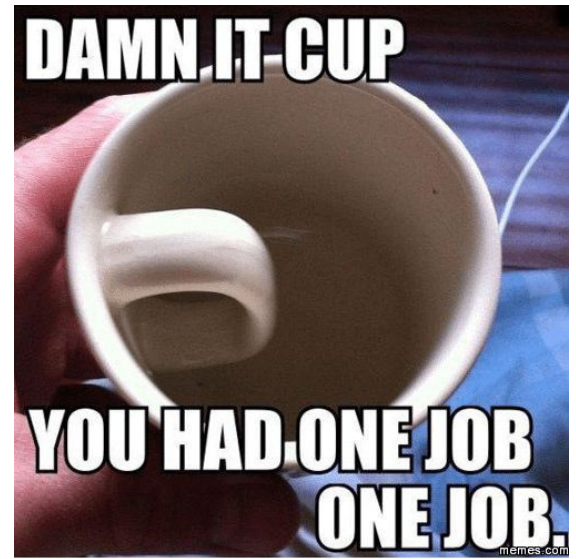
Single Responsibility Principle (SRP)

Every module (or class) should have a single responsibility, and all its methods should contribute to fulfilling that responsibility - move any methods that don't belong to another component

Seemingly contradicts the multiple “responsibilities” of CRC cards, which refer to finer granularity tasks that should all be part of the same focused single purpose. Perhaps better to call it the Single Purpose Principle (SPP)

- ❑ problem: If a class has multiple purposes, then it is more likely to be affected by a requirements change: It knows too much
- ❑ solution: Break up the functionality into multiple classes, where each individual class has only one purpose. Leads to more smaller classes

Greater cohesion, lower coupling, fewer test cases



SRP

Sanity check 1: Can you describe the component's purpose in one sentence without using the word “and”?

Sanity check 2: Does it make sense to say “The <class name> <method name> itself” for each method? (after fixing grammar)

Sanity check 3: Does the component know/do too much?
Recall [code smell “bloaters”](#).

[SRP Example](#)



What are Cohesion and Coupling?

Maximize *Cohesion* = degree to which program unit exhibits a single purpose.
Addressed by Single Responsibility Principle

Minimize *Coupling* = interdependencies among program units, “need to know”.
The degree to which one component C depends on the “innards” of another component D, as opposed to its interface, so that if component D changes internally then component C also needs to change. Addressed by the other SOLID principles. Recall code smell "couplers"



Open-Closed Principle (OCP)

Code units should be “open for extension and closed for modification”, to allow change without modifying any existing code that works because change introduces new bugs (“[regression](#)”)

Seemingly contradicts refactoring, but most [refactoring operations](#) (discussed Thursday) seek to move out parts of the code to leave behind an OCP code unit



OCP

Backwards compatibility



Since requirements change, we need to be able to change existing program behavior

- Change the behavior of an existing class by deriving a new subclass ([inheritance](#))
- Change the behavior of an existing class by composition with a new class ([delegation](#))
- Change the behavior of an existing method by adding a wrapper ([adapter](#)) around that function
- Change the behavior of a library or service by adding new API functions or a new implementation of the same [interface](#)

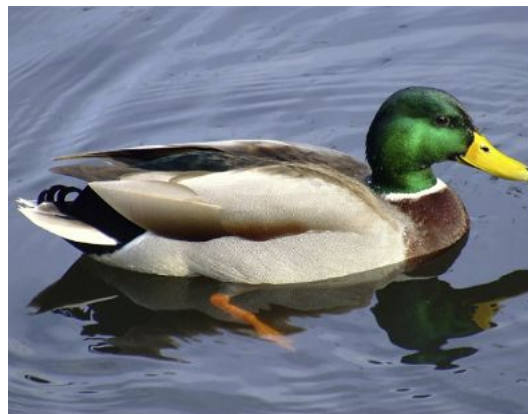
Once upon a time, the conventional way to change the behavior of an existing class was to derive a new subclass, i.e., inheritance, but alternatives are often better

Notice CRC cards have composition (collaborators) but no inheritance - you can still implement using inheritance (in an OO programming language), but be careful about violating LSP and ISP principles!

Liskov Substitution Principle (LSP)

Extends the inheritance variant of OCP to require that subtypes must be substitutable for their base types ([polymorphism](#))

If class A is a subtype of class B , then we should be able to replace B with A without disrupting the behavior of our program



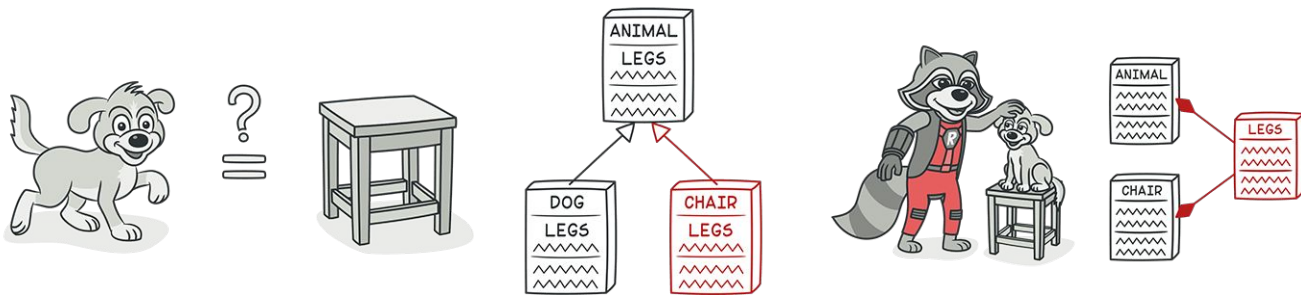
If it looks like a duck and quacks like a duck but it needs batteries, you probably have the wrong abstraction.

LSP

Do inherited methods make sense for instances of the derived class? Do overridden methods take the same parameter types and return the same types as the base class? Do they have side-effects on the same data?

If not, then we are not actually inheriting - instead should delegate any common functionality, and if there is no common functionality then there's no relationship at all so why pretend?

Code already using references to the base classes should be able to use instances of the derived classes without being aware of the switch

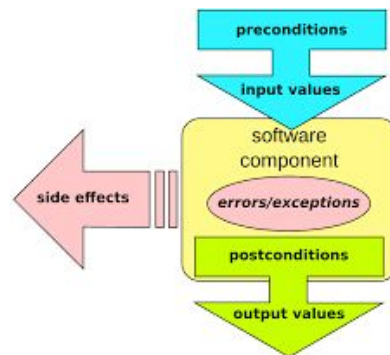


Design by Contract

The contract of a method informs the author of a class about the behaviors that she can safely rely on

The contract could be specified by declaring preconditions and postconditions for each method: The preconditions must be true in order for the method to execute. On completion, after executing the method, the method guarantees that the postconditions are true

Derived classes need to fulfill the same contract.



Interface Segregation Principle (ISP)

CRC cards can be thought of as modeling [interfaces](#) instead of how those interfaces are implemented in code

Components that implement an interface should only have to provide implementations of the methods relevant to them

Clients should not be forced to depend upon interfaces they do not use

Large interfaces should be split into smaller ones



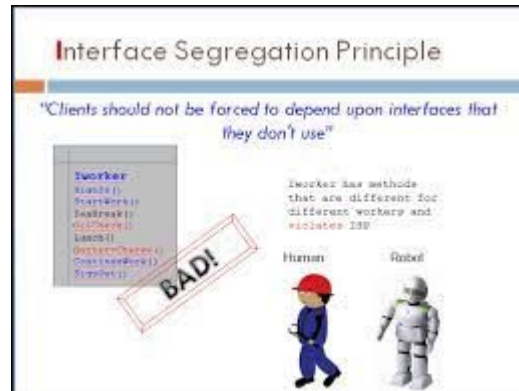
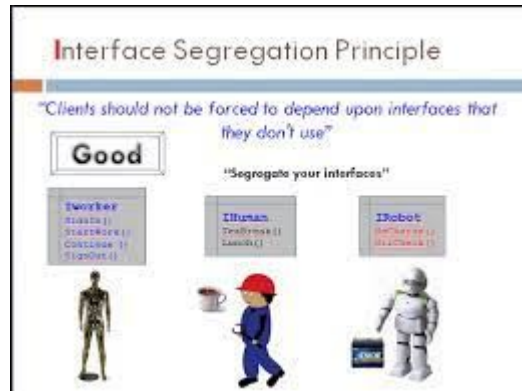
Interface Segregation Principle

If IRequireFood, I want to Eat(Food food) not,
LightCandelabra() or LayoutCutlery(CutleryLayout preferredLayout)

ISP

Analogous to SRP: If an interface has multiple purposes, then it is more likely to be affected by a requirements change

Break up into multiple interfaces, where each individual interface has only one purpose. Leads to more smaller interfaces

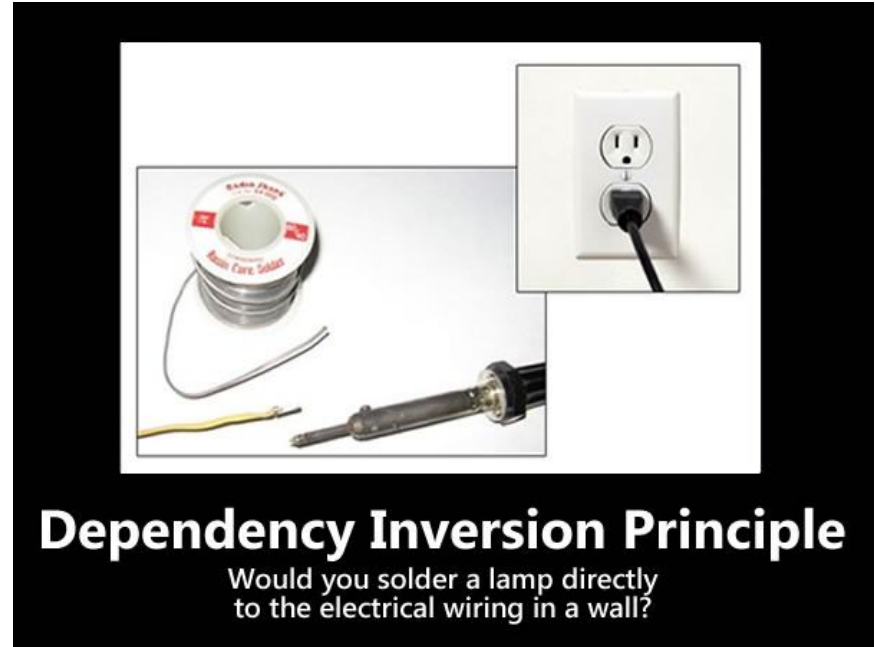


Dependency Inversion Principle (DIP)

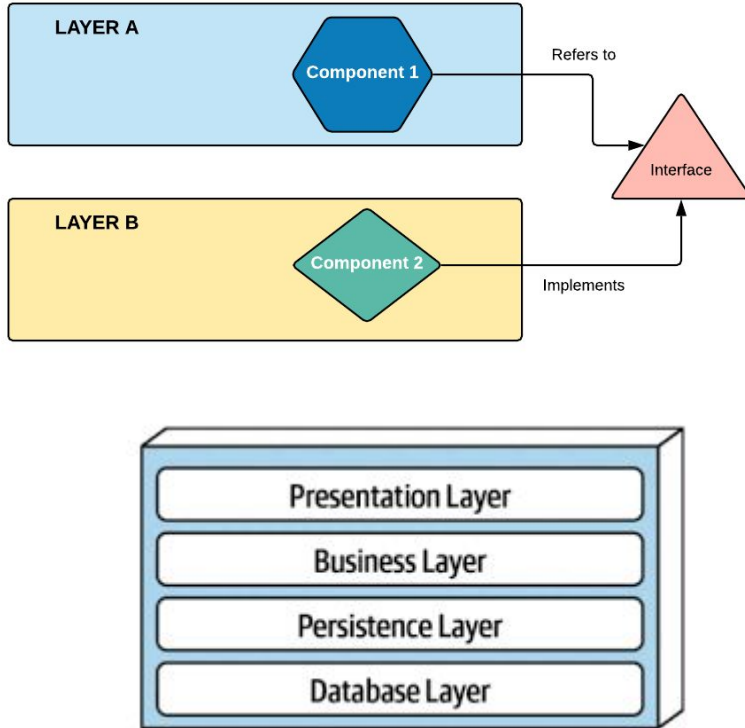
High-level modules should not depend on low-level modules, which makes it hard to reuse the high-level modules

Both should depend on abstractions

Abstractions should not depend on details - details should depend on abstractions



DIP



DIP does not just change the *direction* of the dependency, it *splits* the dependency between the high-level and low-level modules by introducing an abstraction between them

This seemingly contradicts the [layered architectural style](#), where each layer builds on the layer below - the contradiction is resolved by depending on an abstraction of the layer below, not the details of the lower layer

“SOLID” Software Engineering Design Principles

1. **Single Responsibility**
2. **Open/Closed**
3. **Liskov Substitution**
4. **Interface Segregation**
5. **Dependency Inversion**



Single Responsibility Principle

A class should have only a single responsibility (i.e. only one potential change in the software's specification should be able to affect the specification of the class)



Open / Closed Principle

A software module (it can be a class or method) should be open for extension but closed for modification.



Liskov Substitution Principle

Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program.



Interface Segregation Principle

Clients should not be forced to depend upon the interfaces that they do not use.



Dependency Inversion Principle

Program to an interface, not to an implementation.

Don't Repeat Yourself principle (DRY)



Another D - SOLIDD?

Avoid duplicate code and redundant work, instead abstract out common code and put in a single location

Make sure that you only implement each feature and requirement once

DRY is about putting a piece of functionality in one place vs. SRP is about making sure a class does only one thing

DRY

Sanity check: During CRC animation, are there two or more ways to fulfill the same use case or flow?

But don't go too far...

“ARID” = All Readability is Dead

Types of Clones

Original code

```
int main() {  
    int x = 1;  
    int y = x + 5;  
    return y;  
}
```

```
int func1() {  
    int x = 1;  
    int y = x + 5;  
    return y;  
}
```

Exact match

```
int func2() {  
    int p = 1;  
    int q = p + 5;  
    return q;  
}
```

Exact match, with
only the variable
names differing

```
int func3() {  
    int s = 1;  
    int t = s + 5;  
    s++;  
    return t;  
}
```

Near exact match

As defined in an experiment comparing existing clone detection techniques at the 1st International Workshop on Detection of Software Clones (02)

Upcoming Assignments

- [Second Iteration](#) due Thursday, November 30, by 11:59pm
- [Second Iteration Demo](#) due Thursday, December 7, by 11:59pm



Thursday

Refactoring



No Discussion of First Assessment Yet!

Several students have an extended or shifted time period, we will discuss after everyone's deadline has passed



Ask Me Anything